JavaScript is disabled on your browser.

OVERVIEW
PACKAGE

**Java™ Platform**
**Standard Ed. 8**

**PREV CLASS** FRAMES ALL CLASSES
**NEXT CLASS** NO FRAMES

SUMMARY:
NESTED |
ENUM CONSTANTS |
FIELD | METHOD
DETAIL:
ENUM CONSTANTS |
FIELD | METHOD

compact1, compact2, compact3

java.util.concurrent

# Enum TimeUnit

java.lang.Object
  java.lang.Enum<TimeUnit>
    java.util.concurrent.TimeUnit

**All Implemented Interfaces:**

`Serializable, Comparable<TimeUnit>`

---

```
public enum TimeUnit
extends Enum<TimeUnit>
```

A `TimeUnit` represents time durations at a given unit of granularity and provides utility methods to convert across units, and to perform timing and delay operations in these units. A `TimeUnit` does not maintain time information, but only helps organize and use time representations that may be maintained separately across various contexts. A nanosecond is defined as one thousandth of a microsecond, a microsecond as one thousandth of a millisecond, a millisecond as one thousandth of a second, a minute as sixty seconds, an hour as sixty minutes, and a day as twenty four hours.

A `TimeUnit` is mainly used to inform time-based methods how a given timing parameter should be interpreted. For example, the following code will timeout in 50 milliseconds if the `lock` is not available:

```
 Lock lock = ...;
 if (lock.tryLock(50L, TimeUnit.MILLISECONDS)) ...
```

while this code will timeout in 50 seconds:

```
 Lock lock = ...;
 if (lock.tryLock(50L, TimeUnit.SECONDS)) ...
```

Note however, that there is no guarantee that a particular timeout implementation will be able to notice the passage of time at the same granularity as the given `TimeUnit`.

**Since:**

`1.5`

### *Enum Constant Summary*

**Enum Constants**

| Enum Constant | Description |
|---|---|
| `DAYS` | Time unit representing twenty four hours |
| `HOURS` | Time unit representing sixty minutes |
| `MICROSECONDS` | Time unit representing one thousandth of a millisecond |
| `MILLISECONDS` | Time unit representing one thousandth of a second |
| `MINUTES` | Time unit representing sixty seconds |
| `NANOSECONDS` | Time unit representing one thousandth of a microsecond |
| `SECONDS` | Time unit representing one second |

### *Method Summary*

**Concrete Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| `long` | `convert(long sourceDuration, TimeUnit sourceUnit)` | Converts the given time duration in the given unit to this unit. |
| `void` | `sleep(long timeout)` | Performs a `Thread.sleep` using this time unit. |
| `void` | `timedJoin(Thread thread, long timeout)` | Performs a timed `Thread.join` using this time unit. |
| `void` | `timedWait(Object obj, long timeout)` | Performs a timed `Object.wait` using this time unit. |
| `long` | `toDays(long duration)` | Equivalent to `DAYS.convert(duration, this)`. |

|  |  |  |
|---|---|---|
| long | **toHours**(long duration) | Equivalent to `HOURS.convert(duration, this)`. |
| long | **toMicros**(long duration) | Equivalent to `MICROSECONDS.convert(duration, this)`. |
| long | **toMillis**(long duration) | Equivalent to `MILLISECONDS.convert(duration, this)`. |
| long | **toMinutes**(long duration) | Equivalent to `MINUTES.convert(duration, this)`. |
| long | **toNanos**(long duration) | Equivalent to `NANOSECONDS.convert(duration, this)`. |
| long | **toSeconds**(long duration) | Equivalent to `SECONDS.convert(duration, this)`. |
| static **TimeUnit** | **valueOf**(**String** name) | Returns the enum constant of this type with the specified name. |
| static **TimeUnit**[] | **values**() | Returns an array containing the constants of this enum type, in the order they are declared. |

### Methods inherited from class java.lang.Enum

`clone, compareTo, equals, finalize, getDeclaringClass, hashCode, name, ordinal, toString, valueOf`

### Methods inherited from class java.lang.Object

`getClass, notify, notifyAll, wait, wait, wait`

## *Enum Constant Detail*

### NANOSECONDS

```
public static final TimeUnit NANOSECONDS
```

Time unit representing one thousandth of a microsecond

### MICROSECONDS

```
public static final TimeUnit MICROSECONDS
```

Time unit representing one thousandth of a millisecond

### MILLISECONDS

```
public static final TimeUnit MILLISECONDS
```

Time unit representing one thousandth of a second

### SECONDS

```
public static final TimeUnit SECONDS
```

Time unit representing one second

### MINUTES

```
public static final TimeUnit MINUTES
```

Time unit representing sixty seconds

### HOURS

```
public static final TimeUnit HOURS
```

Time unit representing sixty minutes

### DAYS

```
public static final TimeUnit DAYS
```

Time unit representing twenty four hours

## *Method Detail*

### values

```
public static TimeUnit[] values()
```

Returns an array containing the constants of this enum type, in the order they are declared. This method may be used to iterate over the constants as follows:

```
for (TimeUnit c : TimeUnit.values())
    System.out.println(c);
```

**Returns:**

`an array containing the constants of this enum type, in the order they are declared`

### valueOf

```
public static TimeUnit valueOf(String name)
```

Returns the enum constant of this type with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this type. (Extraneous whitespace characters are not permitted.)

**Parameters:**

`name - the name of the enum constant to be returned.`

**Returns:**

`the enum constant with the specified name`

**Throws:**

`IllegalArgumentException - if this enum type has no constant with the specified name`

`NullPointerException - if the argument is null`

### convert

```
public long convert(long sourceDuration,
                    TimeUnit sourceUnit)
```

Converts the given time duration in the given unit to this unit. Conversions from finer to coarser granularities truncate, so lose precision. For example, converting `999` milliseconds to seconds results in `0`. Conversions from coarser to finer granularities with arguments that would numerically overflow saturate to `Long.MIN_VALUE` if negative or `Long.MAX_VALUE` if positive.

For example, to convert 10 minutes to milliseconds, use: `TimeUnit.MILLISECONDS.convert(10L, TimeUnit.MINUTES)`

**Parameters:**

`sourceDuration` - the time duration in the given `sourceUnit`

`sourceUnit` - the unit of the `sourceDuration` argument

**Returns:**

the converted duration in this unit, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

### toNanos

```
public long toNanos(long duration)
```

Equivalent to `NANOSECONDS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

### toMicros

```
public long toMicros(long duration)
```

Equivalent to `MICROSECONDS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

### toMillis

```
public long toMillis(long duration)
```

Equivalent to `MILLISECONDS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

### toSeconds

```
public long toSeconds(long duration)
```

Equivalent to `SECONDS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

### toMinutes

```
public long toMinutes(long duration)
```

Equivalent to `MINUTES.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

**Since:**

1.6

### toHours

```
public long toHours(long duration)
```

Equivalent to `HOURS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration, or Long.MIN_VALUE if conversion would negatively overflow, or Long.MAX_VALUE if it would positively overflow.

**Since:**

1.6

### toDays

```
public long toDays(long duration)
```

Equivalent to `DAYS.convert(duration, this)`.

**Parameters:**

`duration` - the duration

**Returns:**

the converted duration

**Since:**

1.6

### timedWait

```
public void timedWait(Object obj,
                      long timeout)
               throws InterruptedException
```

Performs a timed `Object.wait` using this time unit. This is a convenience method that converts timeout arguments into the form required by the `Object.wait` method.

For example, you could implement a blocking `poll` method (see `BlockingQueue.poll`) using:

```
 public synchronized Object poll(long timeout, TimeUnit unit)
     throws InterruptedException {
   while (empty) {
     unit.timedWait(this, timeout);
     ...
   }
 }
```

**Parameters:**

`obj` - the object to wait on

`timeout` - the maximum time to wait. If less than or equal to zero, do not wait at all.

**Throws:**

`InterruptedException` - if interrupted while waiting

### timedJoin

```
public void timedJoin(Thread thread,
                      long timeout)
               throws InterruptedException
```

Performs a timed `Thread.join` using this time unit. This is a convenience method that converts time arguments into the form required by the `Thread.join` method.

**Parameters:**

`thread` - the thread to wait for

`timeout` - the maximum time to wait. If less than or equal to zero, do not wait at all.

**Throws:**

`InterruptedException` - if interrupted while waiting

### sleep

```
public void sleep(long timeout)
           throws InterruptedException
```

Performs a `Thread.sleep` using this time unit. This is a convenience method that converts time arguments into the form required by the `Thread.sleep` method.

**Parameters:**

`timeout` - the minimum time to sleep. If less than or equal to zero, do not sleep at all.

**Throws:**

`InterruptedException` - if interrupted while sleeping

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.